SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	N. PARNITA	**Reg. No.:**	192525322
Section / Batch:	2025 – 2029	**Date:**	3.8.26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- *Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.*
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1

SECTION A – APPLICATION BASED QUESTIONS

1) Apply the Brute force using Selection Sort.

2) Apply the Brute force using Bubble Sort.

3) Apply the Linear Search to find the element.

Code of Conduct

I N. Parnita *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: N. Parnita

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature:	Signature:	Signature:
Date: 3/8/26	Date:	Date: 3/8/26

1) Apply the Brute Force technique using Selection Sort on the dataset [43, 9, 37, 14, 28, 5].

Given Dataset : [43, 9, 37, 14, 28, 5]

(i) Problem Interpretation

Sort the given numbers in ascending order using the Selection Sort (Brute Force) algorithm by selecting the smallest element in each pass.

(ii) Why Selection Sort ?

* Simple and easy to implement
* Selects the minimum element in every pass.
* Suitable for small datasets.
* Uses fewer swaps.

3) Sorting Process

Pass	Array	Comparisons	Swaps
Initial	43, 9, 37, 14, 28, 5	-	-
Pass 1	5, 9, 37, 14, 28, 43	5	1
Pass 2	5, 9, 37, 14, 28, 43	4	0
Pass 3	5, 9, 14, 37, 28, 43	3	1
Pass 4	5, 9, 14, 28, 37, 43	2	1
Pass 5	5, 9, 14, 28, 37, 43	1	0

Total comparisons : 15
Total swaps : 3

(iv) Time Complexity

* Best case : $O(n^2)$
* Average case : $O(n^2)$
* Worst case : $O(n^2)$
* Space Complexity : $O(1)$

(v) Verification

Final Sorted Array : 5, 9, 14, 28, 37, 43

(vi) Efficiency

Selection sort correctly sorts the array with 15 comparisons and 3 swaps. It is easy to understand and suitable for small datasets but is not efficient for large datasets due to its $O(n^2)$ time complexity.

2) Apply the Brute Force using Bubble Sort on the dataset [34, 12, 29, 45, 18, 7].

1) Problem Interpretation

Sort the given numbers in ascending order using the Bubble Sort algorithm by repeatedly comparing adjacent elements and swapping them if they are in wrong order.

2) Why Bubble Sort?

* Simple and easy to implement
* Compares adjacent elements
* Larger elements move to the end after each pass.
* Suitable for small datasets.

3) Sorting Process

Pass	Array	Comparisons	Swaps
Initial	34, 12, 29, 45, 18, 7	-	-
Pass 1	12, 29, 34, 18, 7, 45	5	4
Pass 2	12, 29, 18, 7, 34, 45	4	2
Pass 3	12, 18, 7, 29, 34, 45	3	1
Pass 4	12, 7, 18, 29, 34, 45	2	1
Pass 5	7, 12, 18, 29, 34, 45	1	1

Total Comparisons : 15
Total swaps : 9

4) Time complexity

* Best case : O(n)
* Average case : $O(n^2)$
* Worst case : $O(n^2)$
* Space Complexity : O(1)

5) Verification

Final sorted Array : 7, 12, 18, 29, 34, 45

6) Efficiency

Bubble sort correctly sorts the datasets with 15 comparisons and 9 swaps. It is easy to understand and suitable for small datasets, but it becomes inefficient for large datasets because of its $O(n^2)$ time complexity.

3) Apply the linear search on the dataset: 22, 31, 47, 58, 16, 39, 64. Apply linear search to find the element 39.

1) Problem Interpretation

Search for the element 39 in the given unsorted dataset using the linear search algorithm. Count the number of comparisons and analyse its time complexity

2) Why linear search?

* Simple and easy to implement
* Works on unsorted data
* Checks each element one by one until the target is found.

3) Search Process

Step	Element Compared	Result
1	22	Not Found
2	31	Not Found
3	47	Not Found
4	58	Not Found
5	16	Not Found
6	39	Found

Position of 39 : 6th position (Index = 5)

Total Comparisons : 6

4) Time Complexity

* Best case : O(1)
* Average case : O(n)
* Worst case : O(n)

5) Verification

The target element 39 is found succesfully after 6 comparisons.

6) Efficiency

Linear search is suitable for small or unsorted datasets. Since 39 is in the latter half of the dataset, more comparisons are required. For large datasets, Linear Search becomes less efficient because it may need to check every element.